1 For Loops and Fancy String Manipulation

Graded Assignment • 30 min

1. Select the expression(s) that produce `True`.

   - [ ] `'do' in "don't"`
   - [ ] `'as' in 'it happens'`
   - [ ] `len('aabbcc') == 3`
   - [ ] `len('aabbcc') == 6`

2. What value does the expression `len('')` produce?

   Note: there are no spaces between the quotes.

   Enter answer here

3. After the following assignment statement has been executed, which expression(s) produce the letter `'g'`?

```
1  dance_style = 'Gangnam'
```

   - [ ] `dance_style[-5]`
   - [ ] `dance_style[3]`
   - [ ] `dance_style[-3]`
   - [ ] `dance_style[-4]`

4. Consider this code:

```
1  title = 'King'
```

   Using `title` and indexing (not slicing), write an expression that produces `'n'`.

   Enter answer here

5. Consider this code:

```
1  s = 'pineapple'
```

   Select the expression(s) that produce $$\color{black}{\verb|'**apple**'|}$$.

   - [ ] `s[5:9]`
   - [ ] `s[4:9]`
   - [ ] `s[-5:-1]`
   - [ ] `s[-5:]`

6. Consider this code:

```
1  vehicle = 'car'
```

   What string does the expression
   `vehicle[-1] + vehicle[1] + vehicle[:1] + 'e' + vehicle` produce?

Enter answer here

7. Select the expression(s) that produce `True`.

- [ ] `'abc123'.isalnum()`
- [ ] `'12.34'.isalnum()`
- [ ] `'abc123'.isdigit()`
- [ ] `'apple'.upper() == 'APPLE'`

8. Select the expression(s) that produce `True` when variable (code)s refers to a `str` that is entirely alphabetic or entirely numeric, and that produce `False` if the `str` is not entirely alphabetic and not entirely numeric.

- [ ] `s.isalpha() or s.isnumeric()`
- [ ] `s.lower() or s.upper() or s.isdigit()`
- [ ] `s.isalpha() and s.isnumeric()`
- [ ] `s.islower() or s.isupper()`

9. Variables `s1` and `s2` refer to `str`s. The expression `s1.find(s2)` returns the index of the first occurrence of `s2 in s1`. The expression `s1.find(s2, 5)` returns the index of the first occurrence of `s2 in s1`, starting at index 5 within `s1`. (See `help(str.find)` for more info)

Write an expression that produces the index of the *second* occurrence of `s2` in `s1`. If `s2` does not occur twice in `s1`, the expression should produce `-1`. Unlike `str.count`, you should allow overlapping occurrences of `s2`.

For example, if `s1` is `'banana'` and `s2` is `'ana'`, your expression should return 3. If `s1` is `'apple'` and `s2` is `'p'`, your expression should return 2.

Your answer must be a single expression that does not use square brackets (string indexing and slicing), and you can only call method `str.find` and use the arithmetic operators (+, -, etc.).

Hint: call `str.find` twice in your expression.

Enter answer here

10. Consider this code:

```
digits = '0123456789'
result = 100

for digit in digits:
    result = result - int(digit)

print(result)
```

What is printed by the code above?

- ◯ 55
- ◯ 100
- ◯ 0
- ◯ 9

11. Consider this code:

```
digits = '0123456789'
result = 0

for digit in digits:
    result = digit

print(result)
```

What is printed by the code above?

- ◯ 45
- ◯ 0
- ◯ 9
- ◯ 0123456789

12. Consider this code:

```
digits = '0123456789'
result = ''

for digit in digits:
    result = result + digit * 2

print(result)
```

What is printed by the code above?

- ○ 0123456789
- ○ 90
- ○ 00112233445566778899
- ○ 45

13. Select the code fragment(s) that print `Happy 30th!`.

☐
```
message = 'Happy 29th!'
new_message = ''

for char in message:
    if char.isdigit():
        new_message = new_message + str((int(char) + 1) % 10)
    else:
        new_message = new_message + char

print(new_message)
```

☐
```
message = 'Happy 29th!'
new_message = ''

for char in message:
    new_message = new_message + str((int(char) + 1) % 10)

print(new_message)
```

☐
```
message = 'Happy 29th!'
new_message = ''

for char in message:
    if not char.isdigit():
        new_message = new_message + char
    else:
        new_message = new_message + str((int(char) + 1) % 10)

print(new_message)
```

☐
```
message = 'Happy 29th!'
new_message = ''

for char in message:
    if char.isdigit():
        new_message = new_message + str((int(char) + 1) % 10)
    new_message = new_message + char

print(new_message)
```

14. Part of the body of the following function is missing. Select the missing code fragment.

```
def common_chars(s1, s2):
    '''(str, str) -> str

    Return a new string containing all characters from s1 that     appear at leastonce in s2.

    >>> common_chars('abc', 'ad')
    'a'
    >>> common_chars('a', 'a')
    'a'
    >>> common_chars('abb', 'ab')
    'abb'
    >>> common_chars('abracadabra', 'ra')
    'araaara'
    '''


    res = ''
```

The characters in the result will appear in the same order as they appear in s1

→ TURN Page

```
    # BODY MISSING

    return res
```

```
for ch in s1:
    if ch in s2:
        res = ch + res
```

```
if ch in s2:
    for ch in s1:
        res = res + ch
```

```
for ch in s2:
    if ch in s1:
        res = res + ch
```

```
for ch in s1:
    if ch in s2:
        res = res + ch
```

2 While Loops, Lists, and Mutability

1. Select the expression(s) that evaluate to `True`.

- [ ] `len('mom') in [1, 2, 3]`
- [ ] `int('3') in [len('a'), len('ab'), len('abc')]`
- [ ] `len([1, 2, 3]) == len(['a', 'b', 'c'])`
- [ ] `[1, 2, 3] in len('mom')`
- [ ] `'3' in [1, 2, 3]`
- [ ] `'a' in ['mom', 'dad']`

2. Consider this code:

```
def secret(s):
    i = 0
    result = ''

    while s[i].isdigit():
        result = result + s[i]
        i = i + 1

    return result
```

Select the function call(s) that result in an error.

- [ ] `secret('123')`
- [ ] `secret('abc123')`
- [ ] `secret('abc')`
- [ ] `secret('123abc')`

3. Consider this code:

```
def example(L):
    """ (list) -> list
    """
    i = 0
    result = []
    while i < len(L):
        result.append(L[i])
        i = i + 3
    return result
```

Which is the best docstring description for function `example`?

- ◯ Return a `list` containing every third *item* from `L` starting at index 0.
- ◯ Return an empty `list`.
- ◯ Return a `list` containing every third *index* from `L` starting at index 0.
- ◯ Return a `list` containing the items from `L` starting from index 0, omitting every third item.

```
def compress_list(L):
    """ (list of str) -> list of str

    Return a new list with adjacent pairs of string elements from L concatenated together starting with indices 0, 1, 2, 3, and so on.

    Precondition: len(L) >= 2 and len(L) % 2 == 0

    >>> compress_list(['a', 'b', 'c', 'd'])
    ['ab', 'cd']
    """
    compressed_list = []
    i = 0

    while i < len(L):
        compressed_list.append(L[i] + L[i + 1])
        # MISSING CODE HERE

    return compressed_list
```

4. Select the missing line of code.

- ◯ `i = i * 2`
- ◯ `i = i + 2`
- ◯ `i = i + i`

- `i = i + 1`

5. What is the sum of the **`odd`** numbers from 1523 through 10503, inclusive? Hint: write a **`while`** loop to accumulate the sum and print it. Then copy and paste that sum. For maximum learning, do it with a **`for`** loop as well, using **`range`**.

Enter answer here

6. Consider this code:

```
def while_version(L):
    """ (list of number) -> number
    """
    i = 0
    total = 0

    while i < len(L) and L[i] % 2 != 0:
        total = total + L[i]
        i = i + 1

    return total
```

The **`while`** loop stops as soon as an even number is found, and the sum of all the previous numbers is returned.

The four functions below use a **`for`** loop to try to accomplish the same task, although they keep iterating through all of the numbers in **`L`** regardless of whether the numbers are even or odd. Only one of them returns the same value as function **`while_version`**. Which one is it?

-
```
def for_version(L):
    found_even = False
    total = 0

    for num in L:
        if num % 2 != 0 and not found_even:
            total = total + num
        else:
            found_even = True

    return total
```

-
```
def for_version(L):
    found_even = False
    total = 0

    for num in L:
        if num % 2 != 0:
            total = total + num
        found_even = True

    return total
```

-
```
def for_version(L):
    found_even = False
    total = 0

    for num in L:
        if num % 2 != 0:
            total = total + num
            found_even = True

    return total
```

-
```
def for_version(L):
    found_even = False
    total = 0

    for num in L:
        if num % 2 != 0:
            total = total + num
        elif not found_even:
            found_even = True

    return total
```

7. Consider this code: 1 point

7. Consider this code:

```
>>> numbers = [1, 4, 3]
>>> # MISSING CODE HERE
>>> print(numbers)[3, 4, 1]
```

Which of the following code fragments(s) could be the missing code in the program above?

- [ ] `numbers = numbers.reverse()`
- [ ] `numbers = reverse(numbers)`
- [ ] `numbers.reverse()`
- [ ] `reverse(numbers)`

8. Consider this code:

```
veggies = ['carrot', 'broccoli', 'potato', 'asparagus']
veggies.insert(veggies.index('broccoli'), 'celery')
print(veggies)
```

What is printed by the code above?

- ○ `['carrot', 'celery', 'broccoli', 'potato', 'asparagus']`
- ○ `['carrot', 'celery', 'potato', 'asparagus']`
- ○ `['carrot', 'broccoli', 'celery', 'potato', 'asparagus']`
- ○ `['celery', 'carrot', 'broccoli', 'potato', 'asparagus']`

9. Your younger sibling has just discovered music from the 1970's. They have put together a playlist of the same 5 songs repeated again and again. Here are the songs:

- ABC by The Jackson 5
- Venus by Shocking Blue
- Lola by the Kinks
- Let It Be by the Beatles
- Cecilia by Simon and Garfunkel

Here is an example of their playlist:

`['Lola', 'Venus', 'Lola', 'Lola', 'Let It Be', 'Lola', 'ABC', 'Cecilia', 'Lola', 'Lola']`

You want to make sure that Lola gets played at most 3 times, so you want to complete this function that edits the playlist:

```
def cap_song_repetition(playlist, song):
    '''(list of str, str) -> NoneType

    Make sure there are no more than 3 occurrences of song in playlist.

    '''
```

Select the loop(s) that accomplish this.

- [ ]
```
while playlist.count(song) > 3:
    playlist.remove(song)
```

- [ ]
```
while playlist.count(song) > 3:
    playlist.pop(playlist.index(song))
```

- [ ]
```
while playlist.count(song) > 3:
    playlist.remove(playlist.index(song))
```

- [ ]
```
while playlist.count(song) >= 3:
    playlist.remove(song)
```

```
while playlist.count(song) > 3:
    playlist.pop(song)
```

10. Consider this code:

```
>>> a = [1, 2, 3]
>>> b = a
>>> # MISSING CODE HERE
>>> print(a, b)
[1, 'A', 3] [1, 'A', 3]
```

Which of the following code fragments(s) could be the missing code in the program above?

- [ ] `b[1] = 'AB'`
- [ ] `a[1] = a[1][0]`
- [ ] `b[-2] = 'A'`
- [ ] `a[1] = 'A'`
- [ ] `a = [1, 'A', 3]`
  `b = [1, 'A', 3]`

11. Consider this code:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> # MISSING CODE HERE
>>> print(a, b)
[1, 'A', 3] [1, 'A', 3]
```

Which of the following code fragments(s) could be the missing code in the program above?

- [ ] `b[1] = 'AB'`
- [ ] `a[1] = a[1][0]`
- [ ] `a = [1, 'A', 3]`
  `b = [1, 'A', 3]`
- [ ] `a[1] = 'A'`
- [ ] `b[-2] = 'A'`

12. Consider this code:

```
def increment_items(L, increment):
    i = 0
    while i < len(L):
        L[i] = L[i] + increment
        i = i + 1

values = [1, 2, 3]
print(increment_items(values, 2))
print(values)
```

What is printed by the program above?

- ( ) `None`
  `[1, 2, 3]`
- ( ) `[3, 4, 5]`
  `None`
- ( ) `[3, 4, 5]`
  `[1, 2, 3]`
- ( ) `None`
  `[3, 4, 5]`

13. Select the code fragment(s) that print `[3, 6, 9]`.

- [ ]
```
values = []
for num in range(3, 9, 3):
    values.append(num)
print(values)
```

- [ ]
```
values = []
for num in range(3, 10, 3):
    values.append(num)
print(values)
```

- [ ]
```
values = []
for num in range(1, 3):
    values.append(num * 3)
print(values)
```

- [ ]
```
values = []
for num in range(1, 4):
    values.append(num * 3)
print(values)
```

14. Select the function calls to **`range`** that, when used to fill in the blank, cause the code to produce the results below.

```
for num in ____________________:
    print(num)
```

The loop should print this:

```
3
11
19
```

- [ ] `range(3, 19, 8)`
- [ ] `range(3, 8, 20)`
- [ ] `range(3, 23, 8)`
- [ ] `range(3, 20, 8)`

3 For Loops Over Indices, Parallel Lists and Strings, and Files

1. Consider this code:

```
def merge(L):
    merged = []
    for i in range(0, len(L), 3):
        merged.append(L[i] + L[i + 1] + L[i + 2])
    return merged

print(merge([1, 2, 3, 4, 5, 6, 7, 8, 9]))
```

What is printed by the code above?

- ◯ `[12, 15, 18]`
- ◯ `[6, 15, 24]`
- ◯ `[123, 456, 789]`
- ◯ `[1, 4, 7]`

2. Consider this code:

```
def mystery(s):
    """ (str) -> bool
    """
    matches = 0
    for i in range(len(s) // 2):
        if s[i] == s[len(s) - 1 - i]: # <--- How many times is        this line reached?
            matches = matches + 1

    return matches == (len(s) // 2)

mystery('civil')
```

Trace the function call **`mystery('civil')`** using the Python Visualizer. How many times is the line marked above reached?

Enter answer here

3. Consider this code:

```
def mystery(s):
    """ (str) -> bool
    """
    matches = 0
    for i in range(len(s) // 2):
        if s[i] == s[len(s) - 1 - i]:
            matches = matches + 1

    return matches == (len(s) // 2)
```

Which is the best docstring description for function **`mystery`**?

- ◯ Return **`True`** if and only if **`s[:len(s) // 2]`** is the same as **`s[len(s) // 2:]`**.
- ◯ Return **`True`** if and only if the number of duplicate characters in **`s`** is equal to **`len(s) // 2`**.
- ◯ Return **`True`** if and only if there are exactly **`len(s) // 2`** characters in **`s`** that are the same character.
- ◯ Return **`True`** if and only if **`s`** is equal to the reverse of **`s`**.

4. In one of the Week 6 lecture videos, we wrote the function **`shift_left`**. Consider this function, which shifts in the other direction:

```
def shift_right(L):
    ''' (list) -> NoneType

    Shift each item in L one position to the rightand shift the    last item to the first position.

    Precondition: len(L) >= 1
    '''

    last_item = L[-1]

    # MISSING CODE GOES HERE

    L[0] = last_item
```

Select the code fragment that correctly completes function **`shift_right`**.

Hint: the correct answer works from the end to the beginning of `L`.

```
for i in range(len(L) - 1):
    L[i] = L[i + 1]
```

```
for i in range(1, len(L)):
    L[len(L) - i] = L[len(L) - i - 1]
```

```
for i in range(len(L)):
    L[i + 1] = L[i]
```

```
for i in range(1, len(L)):
    L[i] = L[i + 1]
```

5. Consider the code (these type contracts get a little tough to write!):

```
def make_pairs(list1, list2):
    ''' (list of str, list of int) -> list of [str, int] list

    Return a new list in which each item is a 2-item list with     the string from thecorresponding position
                                                                                    of list 2

    Precondition: len(list1) == len(list2)

    >>> make_pairs(['A', 'B', 'C'], [1, 2, 3])
    [['A', 1], ['B', 2], ['C', 3]]
    '''

    pairs = []

    # CODE MISSING HERE

    return pairs
```

Select the code fragment(s) that make the function above match its docstring description.

```
for i in range(len(list1)):
    pairs.append([list1[i], list2[i]])
```

```
for i in range(len(list1)):
    inner_list = []
    inner_list.append(list1[i])
    inner_list.append(list2[i])
    pairs.append(inner_list)
```

```
for i in range(len(list1)):
    inner_list = []
    inner_list.append(list1[i])
    inner_list.append(list2[i])
pairs.append(inner_list)
```

```
inner_list = []
for i in range(len(list1)):
    inner_list.append(list1[i])
    inner_list.append(list2[i])
    pairs.append(inner_list)
```

6. Consider this code:

```
values = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Using `values` and indexing with **non-negative indices**, write an expression that evaluates to `5`. Do not use addition, subtraction, or parentheses `( )` (brackets `[]` are required).

Enter answer here

7. Consider this code:

```
breakfast = [['French', 'toast'], ['blueberry', 'pancakes'], ['scrambled', 'eggs']]
```

Using `breakfast` and indexing with **only negative indices**, write an expression that evaluates to `'blueberry'`. Do not use addition, subtraction, or parentheses `( )` (brackets `[ ]` are required).

Enter answer here

8. Consider this code:

```
for i in range(2, 5):
    for j in range(4, 9):
        print(i, j)
```

Trace the code above in the Python Visualizer. How many times is `print(i, j)` executed?

- ◯ 5
- ◯ 24
- ◯ 15
- ◯ 3

9. Consider this code:

```
def contains(value, lst):
    """ (object, list of list) -> bool

    Return whether value is an element of one of the nested        lists in lst.

    >>> contains('moogah', [[70, 'blue'], [1.24, 90, 'moogah'],    [80, 100]])
    True
    """

    found = False  # We have not yet found value in the list.

    # CODE MISSING HERE

    return found
```

Select the code fragment(s) that make the function above match its docstring description.

☐
```
    for sublist in lst:
        if value in sublist:
            found = True
```

☐
```
    for item in lst:
        if value == item:
            value = True
```

☐
```
    for i in range(len(lst)):
```

```
    for j in range(len(lst[i])):
        found = (lst[i][j] == value)
```

```
for i in range(len(lst)):
    for j in range(len(lst[i])):
        if lst[i][j] == value:
            found = True
```

Select the code fragment(s) that make the function above match its docstring.

```
for line in file:
    if letter in line:
        matches.append(line.rstrip('\n'))
```

```
for line in file:
    if line.startswith(letter):
        matches.append(line.rstrip('\n'))
```